EA-0034 - AQELYN Engineering Archive

EA-0034 - AQELYN Machine Identity & Non-Human Identity Governance Engine

Generated: 2026-07-08T20:11:07Z

Publication Standard: FULL_COMPLETE

Master Markdown is the source of truth.

Section 001 - Document Control

Project: AQELYN

Engineering Archive: EA-0034

Implementation Specification: IS-034

Title: AQELYN Machine Identity & Non-Human Identity Governance Engine

Publication Standard: FULL_COMPLETE

Document Type: Master Engineering Archive

Source Format: Markdown

Derived Formats: PDF and HTML

Repository Status: Immutable

Classification: Internal Engineering

Archive Series: EA-0001 through EA-0034

This document is the authoritative engineering specification for the AQELYN Machine Identity and Non-Human Identity Governance Engine. It defines the complete architecture, engineering decisions, interfaces, governance model, operational characteristics, verification strategy, traceability structures, example artifacts, and publication package required for implementation and long-term maintenance within AQELYN.

Authority hierarchy:

1. Project Charter
2. Engineering Principles
3. Repository Standard
4. Architecture Guide
5. Development Rules
6. Publication Standard
7. Engineering Archive

Intended audience: platform architects, security architects, software engineers, identity engineers, DevSecOps engineers, test engineers, release engineers, technical writers, governance specialists, auditors, and maintainers of the AQELYN Cyber Security Operating Environment.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 002 - Revision History

| Version | Date | Archive | Description | Status |

|---|---|---|---|---|

| 1.0 | Initial publication | EA-0034 | First complete engineering archive for IS-034 | Publication candidate |

Engineering change policy: all modifications shall be version controlled, preserve backward traceability, maintain compatibility with previously published Engineering Archives where applicable, be reviewed prior to publication, and be reflected in the Engineering Journal and manifest. No engineering content may be removed without documented rationale and revision record.

Section 003 - Executive Summary

The AQELYN Machine Identity and Non-Human Identity Governance Engine establishes governance over every autonomous identity operating inside the AQELYN Cyber Security Operating Environment. Machine identities include service accounts, workload identities, API identities, cloud managed identities, Kubernetes service accounts, certificates, secrets-bearing principals, CI/CD agents, automation bots, device identities, AI agents, and other non-human actors.

The engine provides a unified governance framework that continuously discovers, inventories, classifies, evaluates, monitors, and tracks these identities through their complete lifecycle. It does not replace identity providers, PKI, secrets managers, cloud IAM services, or CI/CD platforms. Instead, it governs them through standardized integrations and publishes governance intelligence to the AQELYN platform.

Successful implementation enables AQELYN to maintain complete machine identity visibility, detect unmanaged or orphaned identities, govern credentials and certificates, identify privilege accumulation, enforce policy consistently, provide auditable evidence-backed governance decisions, and feed mission-aware risk prioritization across the platform.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 004 - Vision

The engine shall treat machine identities as first-class security entities governed with the same rigor applied to human identities while accounting for their unique operational characteristics. Governance decisions shall be evidence-based, policy-driven, continuously evaluated, and integrated with trust, compliance, lifecycle, and event-driven intelligence.

Architectural principles:

• Universal governance across identity origins and technologies.

• Single source of truth through the AQELYN Universal Object Model.

• Continuous assessment instead of periodic-only audit.

• Evidence-driven findings and decisions.

• Zero Trust alignment: trust is continuously validated and never assumed.

• Lifecycle integrity from request through archival.

• Event-driven operation with immutable publication to the AQELYN Event Bus.

• Platform extensibility through standardized connectors.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 005 - Purpose

The engineering purpose of IS-034 is to provide centralized governance for every autonomous identity in AQELYN. The engine continuously monitors, evaluates, and manages the operational lifecycle of non-human identities to ensure they remain secure, accountable, policy-compliant, and traceable.

Engineering goals include automatic discovery, authoritative inventory, canonical normalization, credential governance, certificate governance, secret governance, ownership validation, governance drift detection, excessive privilege detection, operational trust evaluation, governance policy enforcement, remediation recommendation generation, immutable history, and governance event publication.

Success criteria include reduced orphan identities, improved credential hygiene, fewer certificate-related outages, stronger Zero Trust posture, improved operational visibility, and support for regulatory audit requirements.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 006 - Scope

In scope identity categories include cloud identities such as AWS IAM roles, Azure managed identities, Google Cloud service accounts, and OCI principals; container identities such as Kubernetes service accounts, OpenShift service accounts, operators, controllers, and admission controllers; infrastructure identities such as Windows service accounts, Linux service accounts, scheduled-task accounts, hypervisor accounts, storage controllers, and network appliance identities; application identities such as daemons, microservices, background services, and middleware components; API identities such as OAuth clients, OpenID clients, API keys, JWT issuers, and mutual TLS identities; DevSecOps identities such as CI/CD pipelines, build agents, deployment agents, automation bots, and Git service accounts; secrets and certificate identities; and emerging identity types including AI agents, digital workers, RPA, IoT devices, edge devices, and autonomous systems.

Out of scope: replacing external identity providers, issuing certificates, operating as a secrets manager, replacing enterprise PKI, replacing cloud IAM platforms, replacing SIEM, and replacing PAM. The engine governs and coordinates such systems through standardized integrations.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 007 - Engineering Objectives

OBJ-001 Maintain complete visibility across all machine identities.

OBJ-002 Provide deterministic governance assessments.

OBJ-003 Ensure every identity has accountable ownership.

OBJ-004 Govern every credential lifecycle.

OBJ-005 Govern certificate health and renewal windows.

OBJ-006 Continuously evaluate operational trust.

OBJ-007 Prevent privilege accumulation and toxic combinations.

OBJ-008 Reduce unmanaged identities.

OBJ-009 Provide enterprise-scale governance.

OBJ-010 Support continuous compliance.

OBJ-011 Maintain immutable engineering evidence.

OBJ-012 Support automated remediation workflows.

Engineering KPIs: at least 25 million managed identities; less than 0.5 percent orphan identities; at least 99 percent secret rotation compliance; at least 99.5 percent certificate compliance; at least 99.9 percent governance assessment success; 100 percent evidence linkage; at least 99.9 percent policy evaluation success.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 008 - Definitions and Terminology

Machine Identity: a non-human digital identity used by software, infrastructure, devices, workloads, automation, or autonomous systems to authenticate and interact with systems.

Workload Identity: an identity assigned to a running workload such as a container, virtual machine, serverless function, microservice, or scheduled job.

Governance: the continuous process of establishing, evaluating, enforcing, and auditing policies governing machine identities throughout their lifecycle.

Trust Score: a quantitative representation of confidence in the integrity and behavior of an identity, supplied by the AQELYN Trust Engine.

Governance Score: a deterministic score representing the degree to which a machine identity complies with governance requirements.

Orphan Identity: a machine identity lacking an accountable owner or owning system.

Standing Privilege: persistent privilege assignment that remains active outside an approved operational requirement.

Governance Drift: deviation between an approved governance baseline and current operational state.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 009 - System Context

The engine receives identity telemetry from external providers and internal AQELYN components, normalizes provider objects into canonical AQELYN objects, evaluates governance state, and distributes governance intelligence through Event Bus, Knowledge Graph, Evidence Engine, Workflow Engine, Trust Engine, Mission Engine, Policy Engine, and ISPM Intelligence Engine integrations.

The engine is analytical and governance-focused. It coordinates governance actions and remediation workflows, but it does not directly replace provisioning systems. External platforms remain authoritative for source-specific identity issuance and technical enforcement while AQELYN maintains governance state and evidence.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 010 - Architectural Overview

The architecture is modular and event-driven. Core subsystems include Discovery Manager, Identity Normalization Service, Machine Identity Repository, Lifecycle Management Engine, Governance Assessment Engine, Credential Governance Module, Certificate Governance Module, Trust Integration Adapter, Policy Enforcement Adapter, Recommendation Engine, Event Publisher, Reporting and Analytics Services.

Architectural characteristics: event-driven communication, stateless processing services, pluggable connector framework, canonical data representation via Universal Object Model, immutable event history, evidence-backed governance decisions, horizontal scaling for processing and storage, versioned policies and rule sets, and clear separation of governance, assessment, and orchestration concerns.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 011 - Internal Architecture

Internal processing follows a deterministic pipeline: discovery, normalization, canonical storage, lifecycle evaluation, governance assessment, trust correlation, policy evaluation, recommendation generation, and event publication. Each component has a single responsibility and communicates using AQELYN object and event contracts.

Processing responsibilities:

• Discovery locates machine identities.

• Normalization converts provider objects into canonical AQELYN objects.

• Repository maintains authoritative inventory and history.

• Governance evaluates governance state.

• Lifecycle tracks operational state transitions.

• Policy evaluates rules and constraints.

• Trust consumes trust intelligence.

• Recommendation produces remediation actions.

• Event publication publishes immutable governance events.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 012 - Component Specifications

Discovery Manager orchestrates connectors, scheduled discovery, event-driven discovery, delta synchronization, and discovery health monitoring. Identity Repository stores metadata, provider references, ownership, credentials, certificates, lifecycle state, governance history, evidence references, and trust metadata. Governance Assessment Engine evaluates ownership, credential hygiene, certificate health, authentication, authorization, secret management, operational status, and compliance.

Credential Governance Module monitors credential age, rotation schedules, key length, algorithm strength, secret storage, exposure indicators, and rotation failures. Certificate Governance Module maintains certificate inventory, issuer information, expiration dates, renewal windows, revocation status, chain validation, and trust anchors. Recommendation Engine produces immediate remediation, scheduled remediation, workflow requests, risk prioritization, and mission-aware recommendations.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 013 - Universal Machine Identity Model

Every governed identity extends the AQELYN Universal Object Model. Core attributes include GlobalIdentityId, ProviderId, IdentityType, ProviderType, Environment, Owner, LifecycleState, TrustScore, GovernanceScore, RiskLevel, MissionCriticality, ComplianceStatus, LastAssessment, CreatedAt, UpdatedAt, SourceConnector, EvidenceReferences, PolicyReferences, CredentialReferences, CertificateReferences, and RelationshipReferences.

Supported relationships include Owns, Uses, AuthenticatesTo, Trusts, DependsOn, RunsOn, ManagedBy, RotatedBy, ProtectedBy, AssociatedWith, Invokes, Signs, IssuesTokenFor, MountedBy, and DeployedBy. Relationships are published to the AQELYN Knowledge Graph and remain traceable to source evidence.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 014 - Lifecycle Model

Machine identity lifecycle states: Requested, Approved, Provisioned, Active, CredentialRotation, Maintenance, Suspended, Revoked, Archived. Every transition shall generate an immutable governance event and preserve actor, timestamp, reason, policy references, evidence references, and correlation identifier.

Lifecycle triggers include new discovery, owner assignment, privilege modification, secret rotation, certificate renewal, trust degradation, policy update, mission reassignment, retirement request, risk escalation, stale activity, and governance drift.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 015 - Governance Model

The governance model evaluates six primary domains: ownership, authentication, authorization, credential management, operational health, and compliance. Each domain contributes to an overall Governance Score.

Governance levels: 95-100 Excellent, 85-94 Good, 70-84 Acceptable, 50-69 Elevated Risk, below 50 Critical. Domain weights shall be versioned and configurable through policy while preserving deterministic replay for historical assessments.

Every governance result shall include score, classification, domain contributions, findings, policy references, evidence references, timestamp, rule version, and correlation identifier.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 016 - Trust Model

The engine consumes trust information from the AQELYN Trust Engine. Trust inputs include historical reliability, behavioral consistency, credential integrity, authentication confidence, evidence confidence, identity confidence, runtime integrity, and operational stability.

Trust values influence governance decisions by adjusting priority, remediation urgency, and exception eligibility. Trust shall not override mandatory policy violations unless an explicit policy exception exists and is recorded as evidence.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 017 - Policy Model

Policies are supplied by the AQELYN Policy Engine and evaluated deterministically. Supported categories include ownership policy, credential policy, secret rotation policy, certificate policy, naming convention policy, lifecycle policy, privilege policy, authentication policy, compliance policy, mission policy, exception policy, and connector policy.

Each policy evaluation produces a policy identifier, rule identifier, rule version, evaluation result, evidence references, timestamp, remediation hint, and severity. Policy evolution shall preserve historical replay by retaining rule versions associated with past assessments.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 018 - Event Model

Core event types include MachineIdentityDiscovered, MachineIdentityUpdated, OwnershipAssigned, OwnershipChanged, CredentialRotated, CredentialRotationFailed, CertificateRenewed, CertificateExpiring, GovernanceAssessmentCompleted, GovernanceDriftDetected, PolicyViolationDetected, TrustChanged, RecommendationGenerated, WorkflowRequested, MachineIdentitySuspended, MachineIdentityRevoked, and MachineIdentityArchived.

Standard event envelope fields: EventId, EventType, EventVersion, Timestamp, CorrelationId, SourceEngine, TargetIdentity, Severity, EvidenceReferences, PolicyReferences, TrustContext, MissionContext, DigitalSignature, SchemaVersion, TenantContext, and DataClassification.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 019 - Internal Interface Contracts

Internal interfaces shall be versioned and backward compatible. Primary service contracts include DiscoverIdentities, NormalizeIdentity, StoreIdentity, RetrieveIdentity, EvaluateGovernance, RetrieveTrustScore, EvaluatePolicies, GenerateRecommendation, SubmitRemediation, PublishEvent, LinkEvidence, and UpdateKnowledgeGraph.

Interface evolution shall follow semantic versioning. Breaking changes require explicit migration guidance, compatibility notes, and traceability updates in the Engineering Journal.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 020 - External Interface Contracts

External integration categories include identity providers, cloud platforms, secrets managers, container platforms, DevSecOps platforms, certificate authorities, infrastructure platforms, and emerging autonomous identity providers. Connectors shall isolate provider-specific behavior from core governance logic.

Connector requirements: least-privilege credentials, mutual authentication where supported, retry and backoff, provider rate-limit handling, structured error reporting, schema version declaration, discovery freshness tracking, and connector health events.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 021 - Security Architecture

The engine shall be designed according to Secure-by-Design, Zero Trust, and Defense-in-Depth principles. Primary objectives: prevent unauthorized modification of governance state, protect credential and certificate metadata, ensure integrity of assessments, guarantee authenticity of events, maintain immutable audit history, and minimize engine attack surface.

Security domains: identity security, data security, operational security, supply chain security, platform security, and event security. Trust boundaries: external providers, connector layer, AQELYN internal processing, governance repository, event publication, and administrative interfaces. Each boundary enforces mutual authentication, authorization, input validation, rate limiting where applicable, audit logging, and cryptographic protection.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 022 - Threat Model

Threat categories include unauthorized identity creation, privilege escalation, credential theft, certificate abuse, orphaned identities, governance drift, event tampering, connector compromise, supply chain compromise, and API abuse.

Threat indicators include unexpected privilege assignments, unauthorized owner changes, failed credential rotations, trust degradation, secret reuse, certificate expiration anomalies, connector integrity failures, abnormal authentication patterns, sudden identity creation bursts, and high-risk policy exceptions.

Mitigation strategies include continuous discovery, baseline validation, privilege analytics, secret rotation monitoring, certificate lifecycle governance, ownership validation, drift detection, signed immutable events, connector attestation, software provenance checks, and automated remediation workflows.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 023 - Cryptographic Architecture

The engine shall rely on centrally managed cryptographic services provided by AQELYN and shall not embed proprietary cryptographic implementations. Cryptographic services include digital signatures, hashing, secure random generation, key management integration, certificate validation, and integrity verification.

Every governance assessment shall be associated with assessment identifier, evidence references, policy version, rule version, timestamp, and cryptographic integrity metadata. This supports deterministic replay and independent verification.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 024 - Performance Architecture

Performance targets: at least 25 million inventory records, at least 100 million governance evaluations per hour, at least 10,000 concurrent connector sessions, event publication latency of 250 ms or less, and repository query latency of 200 ms or less at the 95th percentile.

Processing model supports parallel discovery, parallel governance evaluation, incremental synchronization, event-driven reassessment, distributed processing, and horizontal scaling. Performance shall degrade gracefully under resource constraints.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 025 - Scalability Architecture

The engine scales independently across connector execution, governance assessment, repository storage, event publication, analytics, and reporting. Scalability mechanisms include stateless workers, queue-based workload distribution, repository partitioning, read replicas, distributed caches, and elastic worker pools.

Partitioning keys may include tenant, provider, identity type, environment, region, lifecycle state, and risk level. Partitioning strategy shall preserve query efficiency for operational dashboards and audit workflows.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 026 - Reliability and High Availability

Availability objective is 99.9 percent or greater under normal operating conditions. Failure scenarios include connector outage, provider outage, repository failure, event bus interruption, workflow engine unavailability, policy engine timeout, trust engine timeout, and partial network failure.

Recovery strategies include retry policies, circuit breakers, exponential backoff, queue persistence, health monitoring, automatic recovery, event replay, state reconstruction, repository recovery, and disaster recovery procedures. The engine shall support graceful degradation and partial service operation.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 027 - Observability Architecture

Observability shall provide metrics, structured logs, traces, alerts, dashboards, and audit exports. Metrics include discovery throughput, connector health, governance score distribution, trust score distribution, credential rotation status, certificate health, policy violations, recommendation generation rate, event latency, repository growth, queue depth, and workflow completion rate.

Distributed tracing shall cover discovery, normalization, assessment, policy evaluation, recommendation generation, workflow orchestration, evidence linking, knowledge graph update, and event publication.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 028 - Logging Architecture

All operational activities shall generate structured logs with timestamp, correlation id, identity id, event id, component, severity, operation, outcome, duration, evidence reference, policy reference, and tenant context.

Logs support operational troubleshooting, compliance auditing, security investigations, performance analysis, and engineering support. Sensitive values shall not be logged. Secret material, private keys, bearer tokens, and raw credentials are prohibited in logs.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 029 - Monitoring and Operational Metrics

Dashboards shall expose discovery coverage, discovery failures, synchronization latency, governance score trends, high-risk identities, policy compliance, expiring certificates, secret rotation compliance, credential age distribution, connector availability, processing throughput, queue depth, repository capacity, and workflow completion rates.

Alerts shall support configurable thresholds and policy-based escalation. Critical alerts include connector integrity failure, high-volume governance drift, certificate expiration breach, failed rotation above threshold, event publication failure, repository inconsistency, and critical policy violation.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 030 - Engineering Constraints

Architectural constraints: event-driven architecture shall be preserved; Universal Object Model remains canonical; repository structure remains immutable; backward compatibility maintained where applicable; components remain loosely coupled.

Engineering constraints: deterministic rule evaluation, versioned policies, versioned APIs, immutable audit history, traceable engineering decisions, automated tests, reproducible publication builds, and strict separation between source artifact and generated publication artifacts.

Operational constraints: no hard-coded provider dependencies, pluggable connector framework, technology-agnostic implementation, continuous deployment compatibility, least-privilege connectors, and evidence-backed findings.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 031 - Testing Strategy

Testing verifies that every functional and non-functional requirement is implemented correctly, deterministically, and reproducibly. Unit testing covers discovery adapters, normalization, lifecycle transitions, governance scoring, trust calculations, policy evaluation, and recommendation generation. Integration testing covers cloud IAM providers, Kubernetes APIs, secrets managers, certificate authorities, AQELYN Kernel, Event Bus, Policy Engine, Trust Engine, Workflow Engine, Evidence Engine, and Knowledge Graph.

System testing validates end-to-end workflows: discovery, onboarding, assessment, credential rotation, certificate renewal detection, policy violation handling, recommendation generation, event publication, evidence linking, and graph updates. Performance testing evaluates discovery throughput, repository performance, scoring latency, event throughput, and connector concurrency. Security testing validates authorization boundaries, connector authentication, secret protection, certificate validation, event integrity, tamper detection, and audit completeness.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 032 - Validation and Acceptance Criteria

Functional acceptance: all supported machine identity types are discoverable; canonical objects are generated; lifecycle transitions are tracked; governance assessments complete; policy violations are detected; trust integration functions; credential governance detects rotation requirements; certificate governance detects expiration windows; recommendations are generated and prioritized; events are published; evidence is linked.

Non-functional acceptance: availability at least 99.9 percent; assessment determinism 100 percent for identical inputs; event integrity 100 percent verified; evidence linkage 100 percent; policy evaluation at least 99.9 percent successful; repository consistency under defined fault scenarios; no unresolved critical defects; complete traceability.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 033 - Requirements Matrix

| Requirement ID | Description | Priority | Verification |

|---|---|---|---|

| FR-001 | Machine Identity Discovery | Mandatory | Integration Test |

| FR-002 | Identity Classification | Mandatory | Unit Test |

| FR-003 | Ownership Management | Mandatory | Functional Test |

| FR-004 | Lifecycle Tracking | Mandatory | System Test |

| FR-005 | Authentication Assessment | Mandatory | Functional Test |

| FR-006 | Secret Governance | Mandatory | Functional Test |

| FR-007 | Privilege Assessment | Mandatory | Functional Test |

| FR-008 | Trust Evaluation | Mandatory | Integration Test |

| FR-009 | Policy Enforcement | Mandatory | Integration Test |

| FR-010 | Governance Event Publication | Mandatory | Integration Test |

| NFR-001 | Availability >=99.9 percent | Mandatory | Resilience Test |

| NFR-002 | Scalability to >=25M identities | Mandatory | Load Test |

| NFR-003 | Deterministic evaluation | Mandatory | Repeatability Test |

| NFR-004 | Evidence linkage 100 percent | Mandatory | Audit Test |

| NFR-005 | Secure connector operation | Mandatory | Security Test |

Section 034 - Traceability Matrix

| Requirement | Architecture | Component | Verification | Evidence |

|---|---|---|---|---|

| FR-001 | Discovery Layer | Discovery Manager | Integration Test | Discovery Report |

| FR-002 | Normalization | Normalization Service | Unit Test | Mapping Validation |

| FR-003 | Governance Layer | Ownership Manager | Functional Test | Ownership Audit |

| FR-004 | Lifecycle Layer | Lifecycle Engine | System Test | Lifecycle Log |

| FR-005 | Assessment Layer | Governance Engine | Functional Test | Assessment Report |

| FR-006 | Credential Layer | Credential Governance | Functional Test | Credential Audit |

| FR-007 | Authorization Layer | Privilege Analysis | Functional Test | Privilege Report |

| FR-008 | Trust Layer | Trust Adapter | Integration Test | Trust Correlation |

| FR-009 | Policy Layer | Policy Adapter | Integration Test | Policy Evaluation |

| FR-010 | Event Layer | Event Publisher | Integration Test | Event Log |

| NFR-001 | HA Architecture | Health Monitor | Resilience Test | Availability Report |

| NFR-002 | Scalability Architecture | Worker Pool | Load Test | Benchmark Report |

| NFR-003 | Policy Model | Rule Evaluator | Repeatability Test | Determinism Record |

| NFR-004 | Evidence Architecture | Evidence Linker | Audit Test | Evidence Manifest |

| NFR-005 | Security Architecture | Connector Security | Security Test | Security Assessment |

Section 035 - Engineering Journal

Major engineering decisions: canonical machine identity representation; event-driven governance processing; deterministic governance evaluation; trust-aware policy enforcement; immutable governance history; pluggable connector architecture; unified lifecycle management; evidence-backed findings; generated publication artifacts from a master Markdown source.

Design assumptions: identity providers remain authoritative for provisioning; AQELYN governs rather than replaces external IAM platforms; all governance findings are evidence-backed; machine identities continue increasing in volume and diversity; connectors vary in capability and must be isolated from core governance logic.

Open considerations: expanded AI agent identity semantics; confidential computing attestation; hardware root-of-trust integration; post-quantum certificate lifecycle support; cross-domain machine identity federation; real-time secret exposure feeds; richer workload provenance.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 036 - Example Artifacts

Example artifacts included in this package demonstrate expected output structures for engineering and implementation teams. These include a machine identity inventory report, governance assessment report, credential rotation recommendation, certificate expiration alert, ownership compliance report, privilege analysis report, trust correlation report, governance dashboard export, event publication sample, and evidence reference package.

Examples are illustrative and not environment-specific. Implementations shall adapt schemas to the AQELYN Universal Object Model and Universal Event Model while preserving required fields and evidence references.

Implementation Guidance:

• Maintain strict separation between provider-specific connector logic and core governance logic.

• Preserve deterministic behavior by storing policy versions, rule versions, input hashes, and evidence references with every assessment.

• Do not store secret material; store only references, metadata, fingerprints where allowed, and evidence identifiers.

• Use immutable event records for lifecycle transitions and governance decisions.

• Expose clear service boundaries so implementation teams can build, test, and scale each component independently.

Verification Notes:

• Every section requirement shall be translated into unit, integration, system, security, performance, or audit test cases.

• Test evidence shall be retained in the Engineering Archive evidence set.

• Test data shall include cloud, Kubernetes, CI/CD, certificate, and secrets-manager scenarios.

Section 037 - Publication Manifest

The EA-0034 publication package contains Master Markdown, complete PDF, complete HTML, README, manifest.json with SHA-256 hashes, Requirements Matrix, Traceability Matrix, Engineering Journal, example artifacts, Architecture.svg, Component.svg, Workflow.svg, EventFlow.svg, Integration.svg, and final FULL_COMPLETE ZIP.

All derived artifacts are generated from the approved master source. The manifest provides integrity verification for every packaged artifact.

Section 038 - References

Normative references: AQELYN Project Charter, Engineering Principles, Repository Standard, Architecture Guide, Development Rules, Publication Standard, IS-001 through IS-034, EA-0001 through EA-0033.

Informative references: NIST SP 800-207 Zero Trust Architecture; NIST SP 800-53 Security and Privacy Controls; NIST SP 800-63 Digital Identity Guidelines; CIS Controls; OWASP secure software design guidance; CNCF workload identity guidance; SPIFFE/SPIRE workload identity concepts.

Section 039 - Engineering Review

Completeness review: all planned sections completed; requirements addressed; architecture documented; security architecture documented; lifecycle model documented; governance model documented; interfaces documented; testing strategy completed; traceability completed.

Compliance review: repository structure unchanged; engineering sequence maintained; Universal Object Model preserved; event-driven architecture maintained; publication standard satisfied; source-of-truth Markdown established; generated artifacts synchronized to master source.

Section 040 - Publication Readiness

EA-0034 is approved for publication when the following artifacts have been generated from the approved Master Markdown: Master Markdown, PDF, HTML, README, manifest.json, Requirements Matrix, Traceability Matrix, Engineering Journal, example artifacts, five SVG engineering diagrams, and EA-0034_FULL_COMPLETE.zip.

This package is generated as a FULL_COMPLETE publication artifact and is ready for engineering approval before continuing to IS-035.